Lab 14: Deploy a containerized app using ECS Fargate

Phase 1: Create EC2 instance and setup Docker

Step 1: Launch an EC2 Instance

Go to EC2 console -> Launch Instance

Choose:

- Amazon Linux 2 (or Amazon Linux 2023)
- t2.micro / t3.micro (free tier)
- Key pair: select or create one
- Allow SSH (22) from your IP

After launch Connect via SSH.

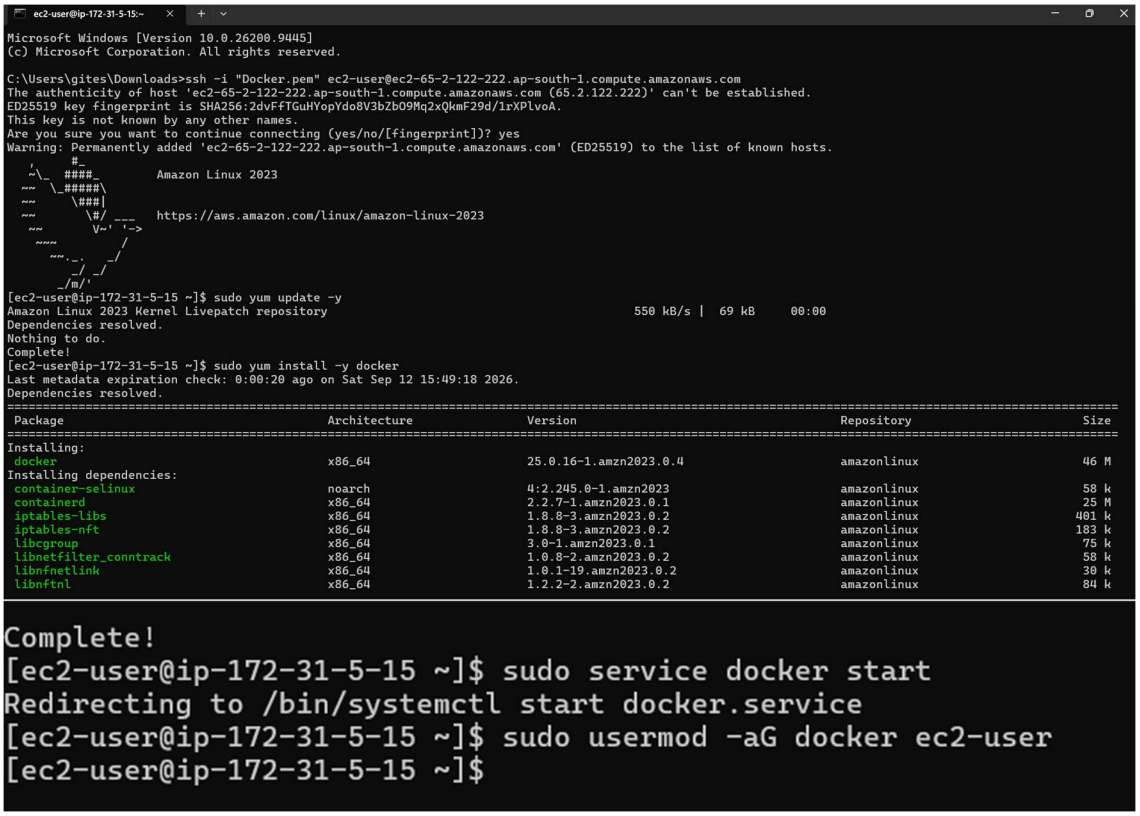
Step 2: Install Docker on your EC2 instance

sudo yum update -y

sudo yum install -y docker

sudo service docker start

sudo usermod -aG docker ec2-user



Step 3: Python Application for ECS Demo

1. Create project folder

- mkdir ecs-demo
- cd ecs-demo

```
[ec2-user@ip-172-31-5-15 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-172-31-5-15 ~]$ mkdir ecs-demo
[ec2-user@ip-172-31-5-15 ~]$ cd ecs-demo
[ec2-user@ip-172-31-5-15 ecs-demo]$
```

2. Create app.py

- nano app.py
  - Paste this

    from flask import Flask
    app = Flask (_name_)
    @app . route (" / ")
    def home() :
        from ECS Fargate python demo ! "
    if __ name__ == “__main__”:
        app.run(host=”0.0.0.0”, port=5000)

3. Create requirements.txt

nano requirements.txt

Flask==3.0.0

```
[ec2-user@ip-172-31-5-15 ecs-demo]$ nano app.py
[ec2-user@ip-172-31-5-15 ecs-demo]$ nano requirements.txt
[ec2-user@ip-172-31-5-15 ecs-demo]$ cat app.py
from flask import Flask
app = Flask (_name_)
@app.route (" / ")
def home() :
        from ECS Fargate python demo ! "
if __name__ == "__main__":
        app.run(host="0.0.0.0", port=5000)
[ec2-user@ip-172-31-5-15 ecs-demo]$ cat requirements.txt
Flask==3.0.0
[ec2-user@ip-172-31-5-15 ecs-demo]$
```

4. Create a Dockerfile

nano Dockerfile

Use official python image

FROM python: 3.11-slim

Set working di rectory

WORKDIR /app

Copy application files

COPY ..

Install dependencies

RUN pip install --no-cache-dir -r requirements.txt

Expose port Flask will run on

EXPOSE 5000

command to run the app

CMD ["python", "app. py"]

```
[ec2-user@ip-172-31-5-15 ecs-demo]$ nano Dockerfile
[ec2-user@ip-172-31-5-15 ecs-demo]$ cat Dockerfile
# Use official python image
FROM python:3.11-slim
# Set working directory
WORKDIR /app
# Copy application files
COPY ..
# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
# Expose port Flask will run on
EXPOSE 5000
# command to run the app
CMD ["python", "app.py"]
[ec2-user@ip-172-31-5-15 ecs-demo]$
```

5. Build the Docker image

Sudo docker build -t ecs-demo-python .

```
[ec2-user@ip-172-31-5-15 ecs-demo]$ sudo docker build -t ecs-demo-python .
[+] Building 9.0s (9/9) FINISHED                                                
```

6. Test locally on EC2

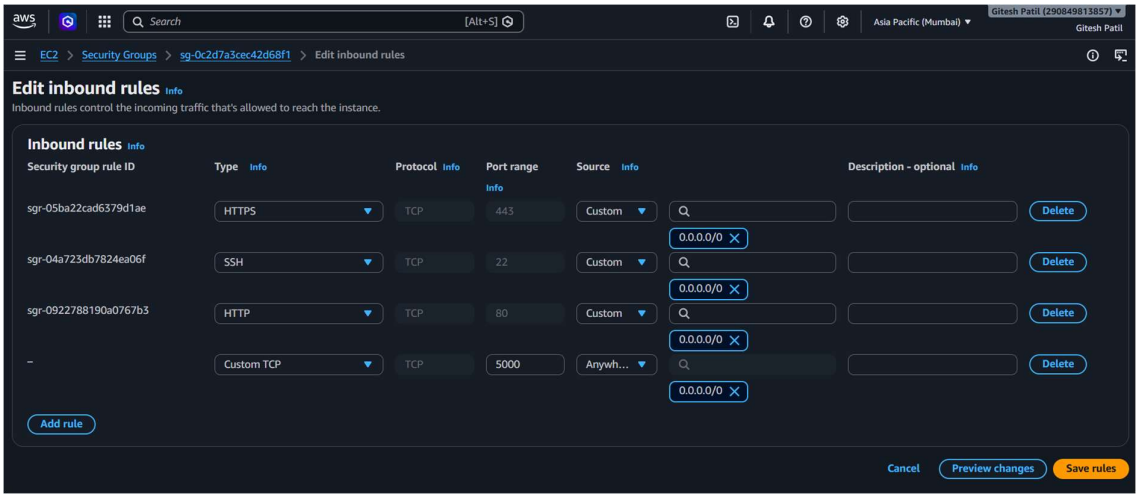
docker run -p 5000: 5000 ecs-demo-python

```
[ec2-user@ip-172-31-5-15 ecs-demo]$ sudo docker run -p 5000:5000 ecs-demo-python
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
```

• Go to the Security Group of EC2 instance on which Docker is installed.

• Modify the Security Group

• Add New Rule å Type: Custom TCP Port: 5050 and Source: Anywhere IP then click on save rules

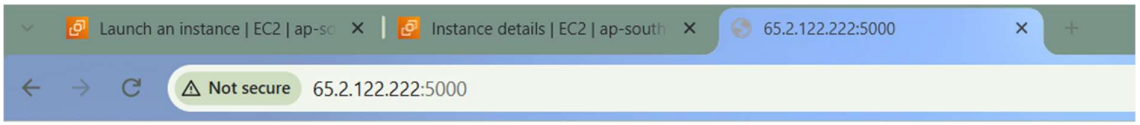


•Open in browser:

http://<EC2 public

You should see:

"Hello from ECS Fargate Python demo!"



Hello from ECS Fargate python demo !

7. Configure User Credentials

Run the command: aws configure

Provide the User Access Key and Secret Access Key

```
[ec2-user@ip-172-31-5-15 ecs-demo]$ aws configure
AWS Access Key ID [****************GG43]: AKIAUHN776VQ265NGG43
AWS Secret Access Key [****************sbP7]: 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
Default region name [ap-south-01]: ap-south-1
Default output format [json]: json
```

aws sts get-caller-identity

```
Default output format [json]: json
[ec2-user@ip-172-31-5-15 ecs-demo]$ aws sts get-caller-identity
{
    "UserId": "AIDAUHN776VQRXMZWFCLB",
    "Account": "290849813857",
    "Arn": "arn:aws:iam::290849813857:user/Gitesh"
}
[ec2-user@ip-172-31-5-15 ecs-demo]$
```

Phase 2: Push to ECR

aws ecr get-login-password --region ap-south-1 \ | docker login --username AWS --password-stdin 290849813857.dkr.ecr.ap-south-1.amazonaws.com

```
[ec2-user@ip-172-31-5-15 ecs-demo]$ aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 759294595201.dkr.ecr.ap-so
uth-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[ec2-user@ip-172-31-5-15 ecs-demo]$
```

Tag:

docker tag ecs-demo-python:latest 290849813857.dkr.ecr.<regi.amazonaws .com/ecs-demo-python:latest

Push:

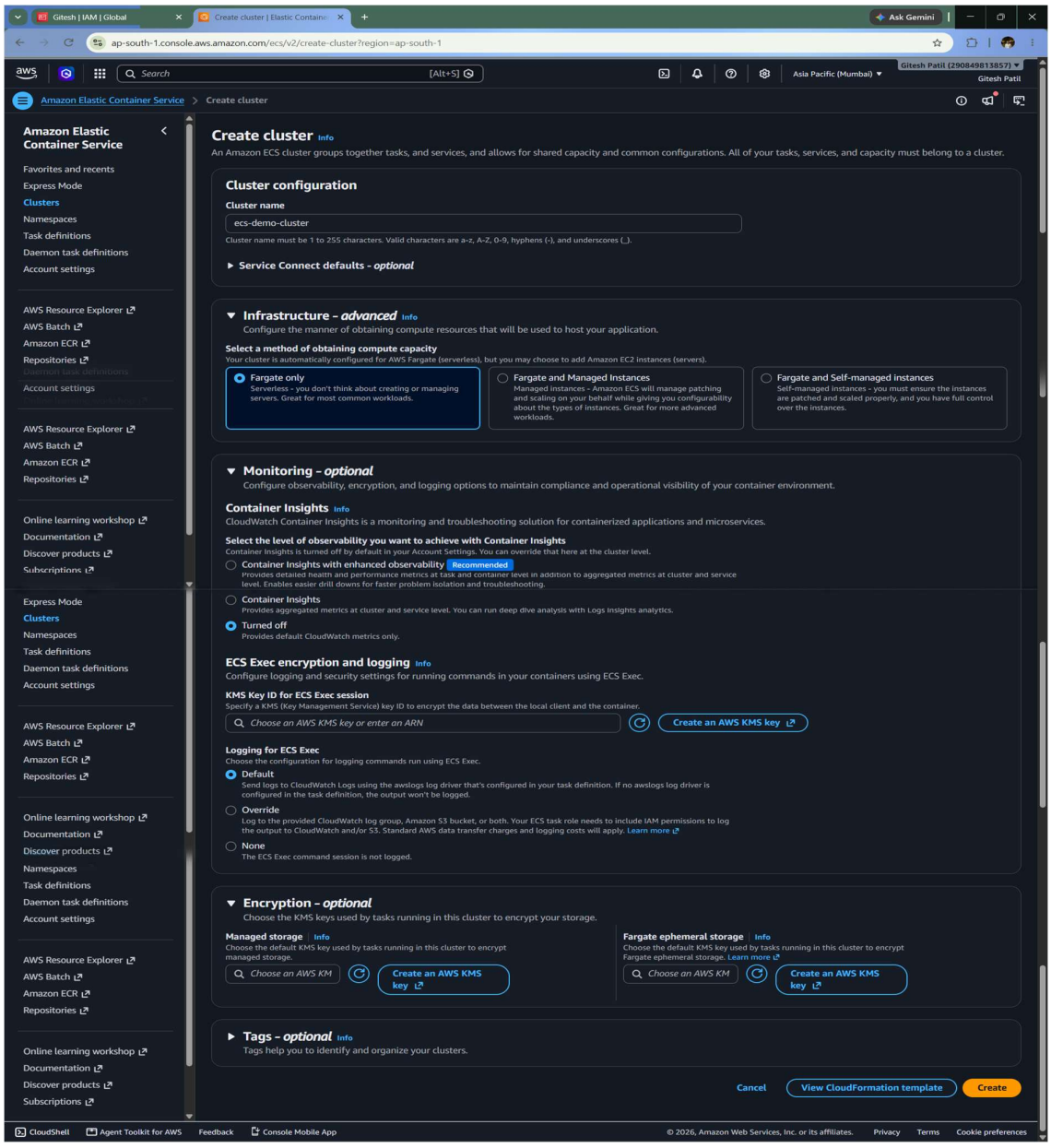
docker push <aws_account_i d> . dkr . ecr . <regi on> . amazonaws . com/ecs-demo-python:latest

```
[ec2-user@ip-172-31-5-15 ecs-demo]$ sudo docker tag ecs-demo-python:latest 290849813857.dkr.ecr.ap-south-1.amazonaws.com/ecs-demo-python:latest
[ec2-user@ip-172-31-5-15 ecs-demo]$ sudo docker push 290849813857.dkr.ecr.ap-south-1.amazonaws.com/ecs-demo-python:latest
The push refers to repository [290849813857.dkr.ecr.ap-south-1.amazonaws.com/ecs-demo-python]
4c655a46fef9: Pushed
960a87ee960b: Pushed
58304b802cc2: Pushed
164ea7718b24: Pushed
7895f75e926a: Pushed
4d43b7f98e4c: Pushed
411a86676185: Pushed
latest: digest: sha256:6ef16e1f8b6cd942417e20ac236ef57bb190b471fd56edf3d52aea6ff99743e0 size: 1783
[ec2-user@ip-172-31-5-15 ecs-demo]$
```

Step 1: Create ECS Cluster

In AWS Console:

1. Go to ECS
2. Left menu Clusters
3. Click Create Cluster
4. Choose Networking only (Fargate)
5. Name it: ecs-demo-cluster
6. Networking:
   - VPC: Select default
   - Subnets: Select all public subnets
   - Security group: Leave default
7. Create cluster.



Step 2: Create Task Definition (Fargate)

1. ECS -> Task Definitions

2. Click Create new Task Definition

3. Select:

- Launch type: FARGATE
- OS: Linux

Name: ecs-demo-python-task

Task Roles: Leave empty (None)

Task Size:

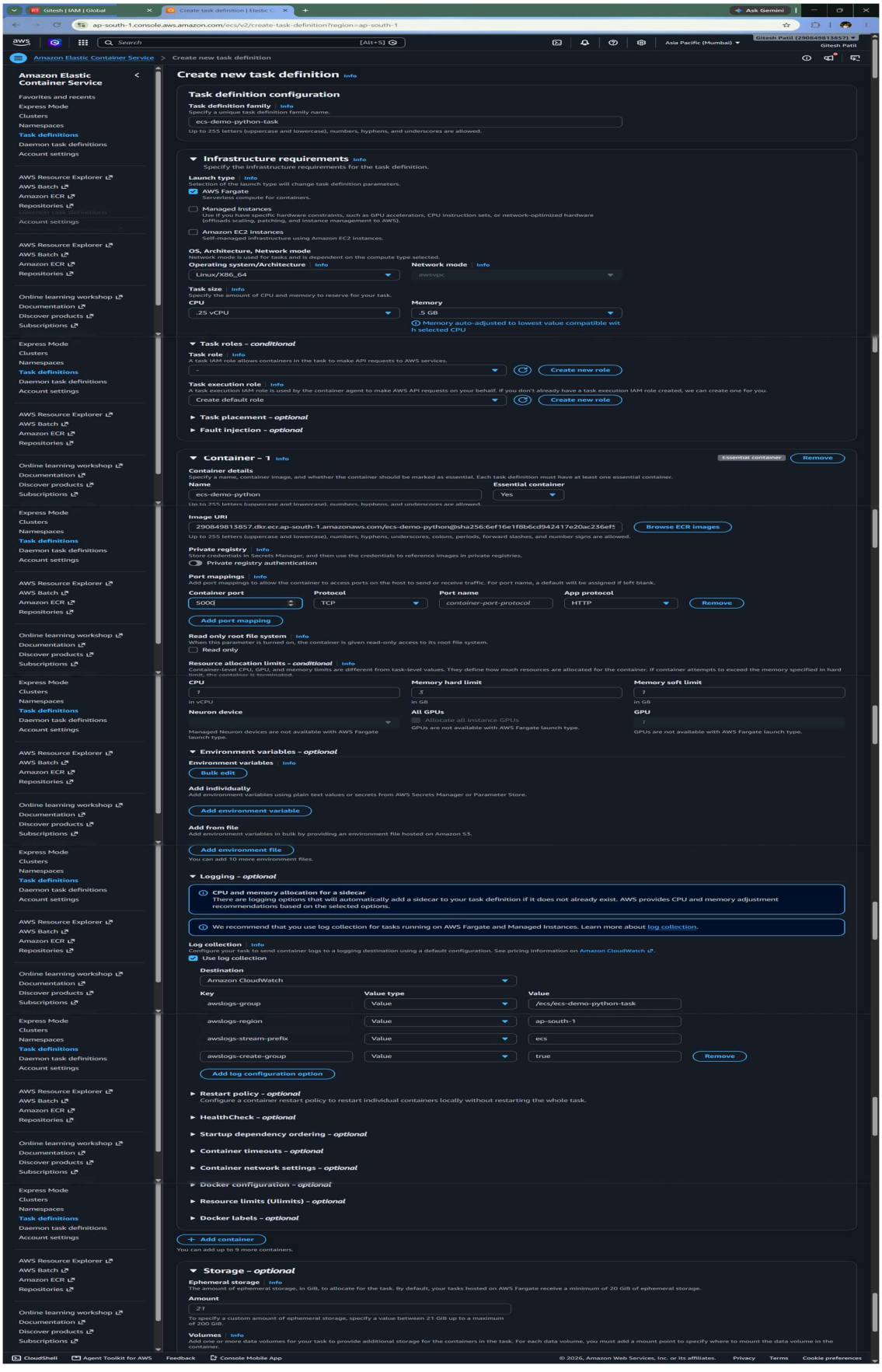
- CPU: 0.25 vCPU (256)
- Memory: 0.5 GB (512 MiB)

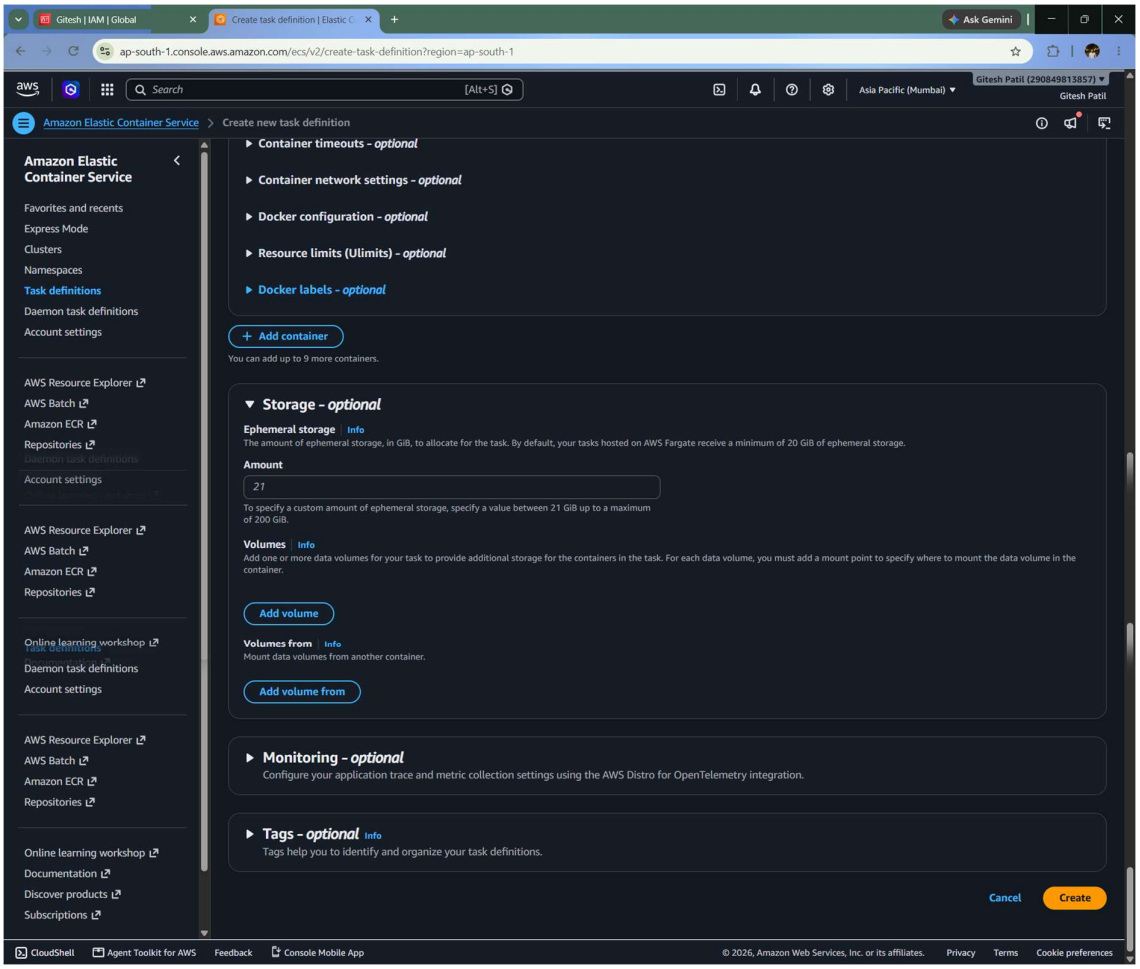
Add Container (most important part)

Click Add container:

- Container name: ecs-demo-python
- Image URI: 759294595201.dkr.ecr.ap-south-1.amazonaws.com/ecs-demo-python:latest
- Port mapping:
  - Container port: 5000
  - Protocol: TCP

Leave everything else default Click Add


Gitesh | IAM | Global
Create task definition | Elastic C
Ask Gemini
ap-south-1.console.aws.amazon.com/ecs/v2/create-task-definition?region=ap-south-1
Search
[Alt+S]
Asia Pacific (Mumbai)
Gitesh Patil
Amazon Elastic Container Service
Create new task definition
Amazon Elastic Container Service
Favorites and recents
Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions
Account settings
AWS Resource Explorer
AWS Batch
Amazon ECR
Repositories
Online learning workshop
Documentation
Discover products
Subscriptions
Create new task definition Info
Task definition configuration
Task definition family Info
Specify a unique task definition family name.
ecs-demo-python-task
Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.
Infrastructure requirements Info
Specify the infrastructure requirements for the task definition.
Launch type Info
Selection of the launch type will change task definition parameters.
AWS Fargate
Serverless compute for containers.
Managed Instances
Use if you have specific hardware constraints, such as GPU accelerators, CPU instruction sets, or network-optimized hardware (offloads scaling, patching, and instance management to AWS).
Amazon EC2 instances
Self-managed infrastructure using Amazon EC2 instances.
OS, Architecture, Network mode
Network mode is used for tasks and is dependent on the compute type selected.
Operating system/Architecture Info
Linux/X86_64
Network mode Info
awsvpc
Task size Info
Specify the amount of CPU and memory to reserve for your task.
CPU
.25 vCPU
Memory
.5 GB
Memory auto-adjusted to lowest value compatible with selected CPU
Task roles – conditional
Task role Info
A task IAM role allows containers in the task to make API requests to AWS services.
Create new role
Task execution role Info
A task execution IAM role is used by the container agent to make AWS API requests on your behalf. If you don't already have a task execution IAM role created, we can create one for you.
Create default role
Create new role
Task placement – optional
Fault injection – optional
Container – 1 Info
Essential container
Remove
Container details
Specify a name, container image, and whether the container should be marked as essential. Each task definition must have at least one essential container.
Name
ecs-demo-python
Essential container
Yes
Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.
Image URI
290849813857.dkr.ecr.ap-south-1.amazonaws.com/ecs-demo-python@sha256:6ef16e1f8b6cd942417e20ac236ef5
Browse ECR images
Up to 255 letters (uppercase and lowercase), numbers, hyphens, underscores, colons, periods, forward slashes, and number signs are allowed.
Private registry Info
Store credentials in Secrets Manager, and then use the credentials to reference images in private registries.
Private registry authentication
Port mappings Info
Add port mappings to allow the container to access ports on the host to send or receive traffic. For port name, a default will be assigned if left blank.
Container port
5000
Protocol
TCP
Port name
container-port-protocol
App protocol
HTTP
Remove
Add port mapping
Read only root file system Info
When this parameter is turned on, the container is given read-only access to its root file system.
Read only
Resource allocation limits – conditional Info
Container-level CPU, GPU, and memory limits are different from task-level values. They define how much resources are allocated for the container. If container attempts to exceed the memory specified in hard limit, the container is terminated.
CPU
1
in vCPU
Memory hard limit
3
in GB
Memory soft limit
1
in GB
Neuron device
Managed Neuron devices are not available with AWS Fargate launch type.
All GPUs
Allocate all instance GPUs
GPUs are not available with AWS Fargate launch type.
GPU
GPUs are not available with AWS Fargate launch type.
Environment variables – optional
Environment variables Info
Bulk edit
Add individually
Add environment variables using plain text values or secrets from AWS Secrets Manager or Parameter Store.
Add environment variable
Add from file
Add environment variables in bulk by providing an environment file hosted on Amazon S3.
Add environment file
You can add 10 more environment files.
Logging – optional
CPU and memory allocation for a sidecar
There are logging options that will automatically add a sidecar to your task definition if it does not already exist. AWS provides CPU and memory adjustment recommendations based on the selected options.
We recommend that you use log collection for tasks running on AWS Fargate and Managed Instances. Learn more about log collection.
Log collection Info
Configure your task to send container logs to a logging destination using a default configuration. See pricing information on Amazon CloudWatch
Use log collection
Destination
Amazon CloudWatch
Key
Value type
Value
awslogs-group
Value
/ecs/ecs-demo-python-task
awslogs-region
Value
ap-south-1
awslogs-stream-prefix
Value
ecs
awslogs-create-group
Value
true
Remove
Add log configuration option
Restart policy – optional
Configure a container restart policy to restart individual containers locally without restarting the whole task.
HealthCheck – optional
Startup dependency ordering – optional
Container timeouts – optional
Container network settings – optional
Docker configuration – optional
Resource limits (Ulimits) – optional
Docker labels – optional
Add container
You can add up to 9 more containers.
Storage – optional
Ephemeral storage Info
The amount of ephemeral storage, in GiB, to allocate for the task. By default, your tasks hosted on AWS Fargate receive a minimum of 20 GiB of ephemeral storage.
Amount
21
To specify a custom amount of ephemeral storage, specify a value between 21 GiB up to a maximum of 200 GiB.
Volumes Info
Add one or more data volumes for your task to provide additional storage for the containers in the task. For each data volume, you must add a mount point to specify where to mount the data volume in the container.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



Step 3: Create ECS Service

Go to: ECS -> Clusters -> ecs-demo-cluster -> Services -> Create

Configure Service

- Launch type: Fargate
- Task Definition: ecs-demo-python-task
- Platform version: LATEST
- Service name: ecs-demo-python-service
    i. Desired tasks: 1
- Click Next

Networking Section

- VPC: default
- Subnets: select public subnets
- Auto-assign public IP: ENABLED (important)
- Security group: Create new

Add inbound rule:

- Type: Custom TCP
- Port: 5000

- Source: 0.0.0.0/0 (for demo access)

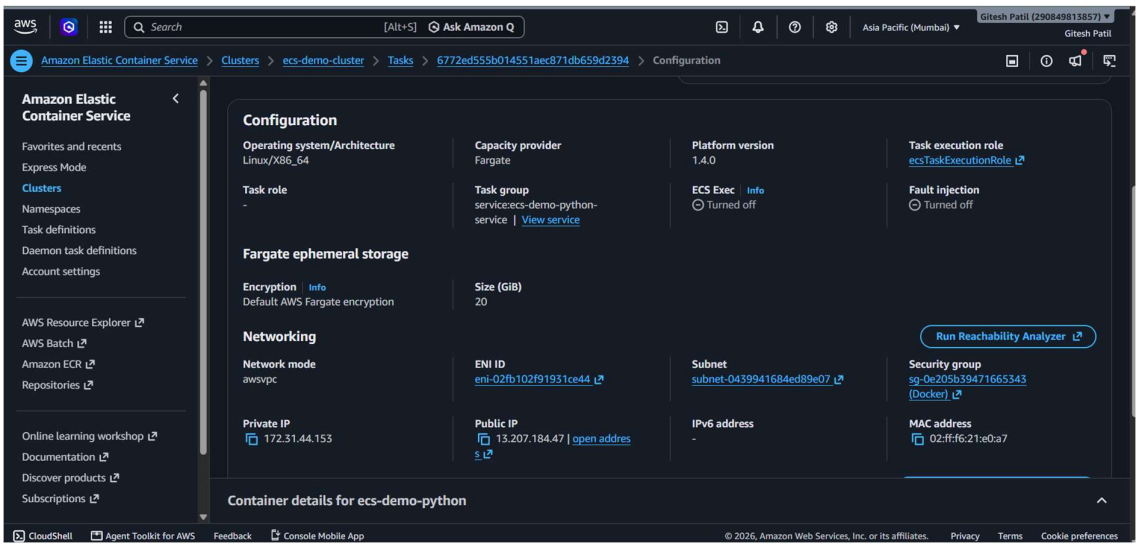
Click Next then Create Service



Step 4: Run & Test Your Task

After creating the service:

1. Go to Clusters -> ecs-demo-cluster
2. Click ecs-demo-python-service
3. Go to Tasks tab
4. Click the running task
5. Scroll to Network section
6. Copy the Public IP



Open in your browser: http://<public-ip>:5000

You should see:"Hello from ECS Fargate Python demo!"

